



TRACKER OPEN API · v1.0 · 2026

# AccuGPS

## *Tracker Open API*

Developer documentation &  
third-party access protocol.

Production · stable

JSON over HTTPS

Token TTL · 72 hours

Base · api.accugps.com

### GETTING STARTED

|   |                |
|---|----------------|
| § | Introduction   |
| § | Authentication |
| § | Conventions    |
| § | Status codes   |
| § | HTTP methods   |

### REPORTS

|     |                           |
|-----|---------------------------|
| GET | Summary report            |
| GET | Trackers state<br>mileage |
| GET | Tracker state<br>mileage  |

### AUTHENTICATION & TRACKERS

|      |                          |
|------|--------------------------|
| POST | User login               |
| GET  | Tracker list             |
| GET  | All trackers<br>location |
| GET  | Tracker location         |
| GET  | Tracker track            |

### REGIONS & EVENTS

|      |                          |
|------|--------------------------|
| GET  | Geo-fence info           |
| GET  | Tracker note (read)      |
| POST | Tracker note (add)       |
| PUT  | Tracker note<br>(update) |
| DEL  | Tracker note<br>(delete) |
| GET  | Tracker alerts           |

# AccuGPS *Tracker Open API*

A token-authenticated REST interface for streaming live tracker telemetry — fleet rosters, real-time and historical locations, trip tracks, fuel and mileage reports, geo-fence configuration, notes and alerts — from AccuGPS-managed devices into your platform.

● Production · stable

● Base URL · <https://api.accugps.com>

● JSON over HTTPS

● Token TTL · 72 hours

## WHAT THIS API DOES

The Open API exposes a curated subset of AccuGPS tracker data for third-party consumption. After exchanging credentials for a short-lived access token, your server sends authenticated requests scoped to either the user's whole fleet or a single device serial number. Each endpoint returns a focused slice of telemetry as JSON.

Use it to mirror fleet status into your dispatch system, build live map dashboards, run summary mileage and fuel reports, manage geo-fence subscriptions, or surface alerts and DTC events into your maintenance workflow.

### 01 Token authentication

One login call returns a JWT valid for 72 hours.

### 02 Fleet & device scoping

List endpoints return all trackers; `{sn}`-scoped paths drill into one device.

### 03 Time-window queries

Historical reads take `start_time` and `end_time` as Unix UTC seconds.

### 04 Stable JSON envelope

`status`, `message`, `data` — consistent across every endpoint.

# Authentication

All requests are authenticated with a bearer token. Exchange your AccuGPS username and password at the [login endpoint](#) to receive an

`access_token`. The token is valid for **three days**; refresh it by re-logging in before expiry.

Pass the token on every subsequent request as the `access_token` HTTP request header. HTTPS is strongly recommended for all traffic.



**Store credentials server-side.** The Tracker

Open API is intended for backend-to-backend integration. Do not embed credentials or long-lived tokens in mobile or browser clients.

REQUEST HEADER

```
{  
  "access_token": "<your_access_token>"  
}
```

# Conventions

|                  |                                                                                             |
|------------------|---------------------------------------------------------------------------------------------|
| Time             | UTC, Unix timestamps in seconds                                                             |
| Coordinates      | Integer milli-degrees – divide latitude / longitude by 3,600,000 to obtain decimal degrees  |
| Empty values     | Strings default to <code>""</code> , numbers default to <code>0</code>                      |
| Parameter casing | All parameters are lowercase snake_case                                                     |
| Versioning       | Every interface carries a version segment in the path – for example <code>/v1/</code>       |
| Response format  | JSON, always with a <code>status</code> / <code>message</code> / <code>data</code> envelope |

# Status codes

HTTP status codes mirror standard REST semantics. Inspect the body's `message` field for human-readable detail.

|                                                                     |                                                                         |                                                                      |                                                             |
|---------------------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------|
| <div></div> <div>200</div> <div>Request successfully answered</div> | <div></div> <div>400</div> <div>Request format or parameter error</div> | <div></div> <div>401</div> <div>User not authenticated</div>         | <div></div> <div>403</div> <div>Forbidden</div>             |
| <div></div> <div>404</div> <div>Not found</div>                     | <div></div> <div>408</div> <div>Request timeout</div>                   | <div></div> <div>429</div> <div>Rate limit — frequent requests</div> | <div></div> <div>500</div> <div>Internal server error</div> |

# HTTP methods

GET

**Query** resources from the server.

POST

**Create** a new entry on the server.

PUT

**Modify** an existing entry on the server.

DELETE

**Remove** an entry from the server.

# Endpoints

Authentication, fleet rosters, real-time and historical location, mileage and summary reports, geo-fence configuration, notes and alerts.

# User login

Exchange a username and password for a short-lived `access_token` .

**POST** /api/open/v1/login

## BODY PARAMETERS

| PARAMETER | TYPE   | REQUIRED | DESCRIPTION            |
|-----------|--------|----------|------------------------|
| username  | string | required | Account email address. |
| password  | string | required | Account password.      |

## RESPONSE FIELDS

| FIELD             | TYPE   | DESCRIPTION                                             |
|-------------------|--------|---------------------------------------------------------|
| status            | int    | HTTP status code mirrored in the body.                  |
| message           | string | Human-readable result.                                  |
| data.access_token | string | JWT to send on subsequent requests. Valid for 72 hours. |

REQUEST

**POST** /api/open/v1/login

Content-Type: application/json

```
{
  "username": "you@example.com",
  "password": "<your_password>"
}
```

200 RESPONSE

```
{
  "status": 200,
  "message": "Operation is successful.",
  "data": {
    "access_token": "<your_access_token>"
  }
}
```



# Tracker list

Return every tracker the authenticated user owns or has shared access to, along with current online status and last-known telemetry.

**GET** /api/open/v1/trackers

## PARAMETERS

No parameters. The fleet returned is scoped to the authenticated user.

## RESPONSE FIELDS

### EXAMPLE REQUEST

**GET** /api/open/v1/trackers

### 200 RESPONSE

```
{
  "status": 200,
  "message": "Operation is successful.",
  "data": [
    {
      "id": "55555555555555555555555555555555",
      "login": 1,
      "sn": "SIMULATOR0",
      "iccid": "55555555555555555555555555555555",
      "alias": "SIMULAT",
      "out_of_service": 0,
      "is_shared": 0,
      "is_moving": 1,
      "power_saving": 0,
      "charging_status": 1,
      "altitude": 10,
      "longitude": 0,
      "latitude": 0,
      "degree": 270.5,
      "speed": 20,
      "temp": 0,
      "fuel_city": 10.17316,
      "fuel_highway": 8.04795,
      "gps": 31,
      "gsm": 0,
      "pbat": 0,
      "vin": "1GC...559"
    }
  ]
}
```

| FIELD                               | TYPE                | DESCRIPTION                                                        |
|-------------------------------------|---------------------|--------------------------------------------------------------------|
| <code>data[].id</code>              | <code>string</code> | Tracker identifier.                                                |
| <code>data[].login</code>           | <code>int</code>    | <code>0</code> = offline · others online.                          |
| <code>data[].sn</code>              | <code>string</code> | Device serial number.                                              |
| <code>data[].iccid</code>           | <code>string</code> | SIM ICCID.                                                         |
| <code>data[].alias</code>           | <code>string</code> | User-assigned alias.                                               |
| <code>data[].out_of_service</code>  | <code>int</code>    | <code>0</code> = perfect · <code>1</code> = bad.                   |
| <code>data[].is_shared</code>       | <code>int</code>    | <code>0</code> = owned · <code>1</code> = shared.                  |
| <code>data[].is_moving</code>       | <code>int</code>    | <code>0</code> = not moving · <code>1</code> = moving.             |
| <code>data[].power_saving</code>    | <code>int</code>    | <code>0</code> = not power saving · <code>1</code> = power saving. |
| <code>data[].charging_status</code> | <code>int</code>    | <code>0</code> = not charging · <code>1</code> = charging.         |
| <code>data[].altitude</code>        | <code>int</code>    | Altitude.                                                          |
| <code>data[].longitude</code>       | <code>float</code>  | Last-known longitude.                                              |
| <code>data[].latitude</code>        | <code>float</code>  | Last-known latitude.                                               |
| <code>data[].degree</code>          | <code>float</code>  | Heading in degrees.                                                |
| <code>data[].speed</code>           | <code>int</code>    | Speed.                                                             |
| <code>data[].temp</code>            | <code>int</code>    | Temperature.                                                       |
| <code>data[].fuel_city</code>       | <code>float</code>  | City fuel consumption (L / 100 km).                                |

| FIELD                            | TYPE                | DESCRIPTION                            |
|----------------------------------|---------------------|----------------------------------------|
| <code>data[].fuel_highway</code> | <code>float</code>  | Highway fuel consumption (L / 100 km). |
| <code>data[].gps</code>          | <code>int</code>    | GPS signal value.                      |
| <code>data[].gsm</code>          | <code>int</code>    | GSM signal intensity.                  |
| <code>data[].pbat</code>         | <code>int</code>    | Battery percentage.                    |
| <code>data[].vin</code>          | <code>string</code> | Vehicle identification number.         |

# All trackers location

Return the latest location for every tracker under the authenticated user — useful for powering a fleet-wide live map.

**GET** /api/open/v1/trackers/location

## PARAMETERS

No parameters.

## RESPONSE FIELDS

| FIELD                         | TYPE   | DESCRIPTION                          |
|-------------------------------|--------|--------------------------------------|
| data[].id                     | string | Tracker identifier.                  |
| data[].sn                     | string | Device serial number.                |
| data[].location               | object | Location payload.                    |
| data[].location.latitude      | float  | Latitude.                            |
| data[].location.longitude     | float  | Longitude.                           |
| data[].location.degree        | float  | Heading in degrees.                  |
| data[].location.name          | string | Reverse-geocoded address.            |
| data[].location.speed         | int    | Speed in m/s.                        |
| data[].location.location_time | int    | Sample timestamp — Unix UTC seconds. |

**EXAMPLE REQUEST**

**GET** /api/open/v1/trackers/location

**200 RESPONSE**

```
{
  "status": 200,
  "message": "Operation is successful.",
  "data": [
    {
      "id": "55555555555555555855",
      "sn": "SIMULATOR0",
      "location": {
        "longitude": 113.56868,
        "latitude": 22.37418,
        "degree": 0,
        "name": "",
        "speed": 10,
        "location_time": 1474449353
      }
    }
  ]
}
```

# Tracker location

Return the latest location for a single tracker by serial number.

GET

/api/open/v1/trackers/{sn}/location

PATH PARAMETERS

| PARAMETER | TYPE   | DESCRIPTION            |
|-----------|--------|------------------------|
| sn        | string | Tracker serial number. |

RESPONSE FIELDS

| FIELD              | TYPE   | DESCRIPTION                          |
|--------------------|--------|--------------------------------------|
| data.latitude      | float  | Latitude.                            |
| data.longitude     | float  | Longitude.                           |
| data.degree        | float  | Heading in degrees.                  |
| data.name          | string | Reverse-geocoded address.            |
| data.speed         | int    | Speed in m/s.                        |
| data.location_time | int    | Sample timestamp — Unix UTC seconds. |

EXAMPLE REQUEST

GET

/api/open/v1/trackers/3842402CD3/location

200 RESPONSE

```
{
  "status": 200,
  "message": "Operation is successful.",
  "data": {
    "longitude": 113.56868,
    "latitude": 22.37418,
    "degree": 0,
    "name": "",
    "speed": 10,
    "location_time": 1474449353
  }
}
```

# Tracker track

Return the full trip track for a single device over a time window — position segments, stop points, over-speed events, manual notes and aggregate totals.

GET

/api/open/v1/trackers/{sn}/track

PATH PARAMETERS

| PARAMETER | TYPE   | DESCRIPTION            |
|-----------|--------|------------------------|
| sn        | string | Tracker serial number. |

QUERY PARAMETERS

| PARAMETER  | TYPE | REQUIRED | DESCRIPTION                                                     |
|------------|------|----------|-----------------------------------------------------------------|
| start_time | int  | required | Query begin time — Unix UTC seconds.                            |
| end_time   | int  | required | Query end time — Unix UTC seconds.<br>Defaults to current time. |

RESPONSE FIELDS

EXAMPLE REQUEST

GET  
/api/open/v1/trackers/3842402CD3/track?  
start\_time=1494864000  
&end\_time=1494950399

| FIELD                     | TYPE  | DESCRIPTION                                                                                                                                                                     |
|---------------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| over_speeds[][]           | array | Over-speed events grouped per trip. Each point carries latitude , longitude , over_speed_threshold , over_speed_threshold_us , speed , timestamp .                              |
| segments[][]              | array | Position points grouped per trip. Each point includes altitude , degree , driving_time , gps , gsm , latitude , longitude , name , pacc , pbat , speed , temp , location_time . |
| stopping_points[]         | array | Stop points with start_driving_time , start_time , distance and the same point fields as segments .                                                                             |
| notes[]                   | array | Manual notes pinned along the trip. Each carries note , latitude , longitude , name , timestamp and opt ( 1 = add · 2 = update · 3 = delete).                                   |
| total.total_distance      | float | Total driving distance (km).                                                                                                                                                    |
| total.total_driving_time  | float | Total driving time (seconds).                                                                                                                                                   |
| total.total_stopping_time | int   | Total stop time (seconds).                                                                                                                                                      |

200 RESPONSE

```
{
  "status": 200,
  "message": "Operation
is successful.",
  "data": {
    "hash_":
"8c815d44480c45229f3e9c6
764abc4b5",
    "over_speeds": [
      [
        {
          "latitude":
22.37,
          "longitude":
113.59008,

          "over_speed_threshold":
25,

          "over_speed_threshold_us
": 16,

          "speed": 51,
          "timestamp":
1467109512
        }
      ]
    ],
    "segments": [
      [
        {
          "altitude":
75,

          "degree": 0,

          "driving_time": 60,
          "end_time":
1465271041,
          "gps": 8,
          "gsm": 63,
          "latitude":
22.37441,
          "longitude":
113.56866,
          "name": "",
          "pacc": 43,
          "pbat": 84,
          "speed": 3,
          "temp": 27,

          "location_time":
1465271041
        }
      ]
    ],
    "stopping_points": [
```

```
{
  "latitude":
22.37441,
  "longitude":
113.56866,

  "start_driving_time":
1465271041,
  "start_time":
1465271041,
  "distance": 0
}
],
"notes": [
  {
    "note": "note1",
    "latitude":
22.37441,
    "longitude":
113.56866,
    "name": "",
    "timestamp":
1582621353,
    "opt": 1
  }
],
"total": {
  "total_distance":
173.5,

  "total_driving_time":
270060,

  "total_stopping_time":
9205260
}
}
```



# Summary report

Aggregate driving, fuel, stop and speed-bucket statistics for one tracker or the whole fleet, bucketed by month or year.

GET

/api/open/v1/reports/summaries

## QUERY PARAMETERS

| PARAMETER       | TYPE   | REQUIRED | DESCRIPTION                                                   |
|-----------------|--------|----------|---------------------------------------------------------------|
| start_time      | int    | required | Query start time — Unix UTC seconds.                          |
| end_time        | int    | required | Query end time — Unix UTC seconds. Defaults to current time.  |
| sn              | string | required | Device serial number — pass <code>-1</code> for all trackers. |
| statistics_type | int    | required | <div>1 = year ·<br/>2 = month.</div>                          |

## RESPONSE FIELDS

EXAMPLE REQUEST

GET

/api/open/reports/summaries  
?start\_time=1494864000  
&end\_time=1494950399  
&statistics\_type=3  
&sn=-1

200 RESPONSE

```
{
  "status": 200,
  "message": "Operation is
successful.",
  "data": [
    {
      "alias": "SIMULAT",
      "distance": 0,
      "driving_time": 0,
      "fuel": 0,
      "stops": 0,
      "stopsless30min": 0,
      "stopsless2hour": 0,
      "averspeed": 0,
      "maxspeed": 0,
      "speedless10": 0,
      "speedless30": 0,
      "speedless70": 0
    }
  ]
}
```

| FIELD                              | TYPE                | DESCRIPTION                           |
|------------------------------------|---------------------|---------------------------------------|
| <code>data[].alias</code>          | <code>string</code> | Tracker alias.                        |
| <code>data[].distance</code>       | <code>float</code>  | Driving distance (km).                |
| <code>data[].driving_time</code>   | <code>int</code>    | Driving time (seconds).               |
| <code>data[].fuel</code>           | <code>float</code>  | Fuel (L / 100 km).                    |
| <code>data[].stops</code>          | <code>int</code>    | Total number of stops.                |
| <code>data[].stopsless30min</code> | <code>int</code>    | Stops shorter than 30 minutes.        |
| <code>data[].stopsless2hour</code> | <code>int</code>    | Stops between 30 minutes and 2 hours. |
| <code>data[].averspeed</code>      | <code>int</code>    | Average speed.                        |
| <code>data[].maxspeed</code>       | <code>int</code>    | Maximum speed.                        |
| <code>data[].speedless10</code>    | <code>float</code>  | % of time under 10 km/h.              |
| <code>data[].speedless30</code>    | <code>float</code>  | % of time between 10 – 30 km/h.       |
| <code>data[].speedless70</code>    | <code>float</code>  | % of time between 30 – 70 km/h.       |

# Trackers state mileage

Aggregate per-state or per-province driving distance across the entire fleet, useful for fuel-tax (IFTA) and jurisdictional reporting.

GET

/api/open/v1/trackers/state\_mileage

QUERY PARAMETERS

| PARAMETER  | TYPE | REQUIRED | DESCRIPTION                                                  |
|------------|------|----------|--------------------------------------------------------------|
| start_time | int  | required | Query begin time — Unix UTC seconds.                         |
| end_time   | int  | required | Query end time — Unix UTC seconds. Defaults to current time. |

RESPONSE FIELDS

| FIELD                     | TYPE   | DESCRIPTION            |
|---------------------------|--------|------------------------|
| data[].state              | string | State / province code. |
| data[].trackers[].alias   | string | Tracker alias.         |
| data[].trackers[].mileage | float  | State mileage (km).    |

EXAMPLE REQUEST

GET /api/open/v1/trackers/state\_mileage?start\_time=1494864000&end\_time=1494950399

200 RESPONSE

```
{
  "status": 200,
  "message": "Operation is successful.",
  "data": [
    {
      "state": "ON",
      "trackers": [
        { "alias": "A598A0102D",
"mileage": 357.62 },
        { "alias": "A598A0189B",
"mileage": 4163.27 }
      ]
    },
    {
      "state": "QC",
      "trackers": [
        { "alias": "A598A0102D",
"mileage": 114.27 },
        { "alias": "A598A0189B",
"mileage": 0 }
      ]
    }
  ]
}
```

# Tracker state mileage

Per-state driving distance for a single tracker over a time window.

GET

/api/open/v1/trackers/{sn}/state\_mileage

PATH PARAMETERS

| PARAMETER | TYPE   | DESCRIPTION            |
|-----------|--------|------------------------|
| sn        | string | Tracker serial number. |

QUERY PARAMETERS

| PARAMETER  | TYPE | REQUIRED | DESCRIPTION                                                  |
|------------|------|----------|--------------------------------------------------------------|
| start_time | int  | required | Query begin time — Unix UTC seconds.                         |
| end_time   | int  | required | Query end time — Unix UTC seconds. Defaults to current time. |

RESPONSE FIELDS

| FIELD                     | TYPE   | DESCRIPTION            |
|---------------------------|--------|------------------------|
| data[].state              | string | State / province code. |
| data[].trackers[].alias   | string | Tracker alias.         |
| data[].trackers[].mileage | float  | State mileage (km).    |

EXAMPLE REQUEST

GET

/api/open/v1/trackers/A598A0102D/state\_mileage?start\_time=1494864000&end\_time=1494950399

200 RESPONSE

```
{
  "status": 200,
  "message": "Operation is successful.",
  "data": [
    {
      "state": "ON",
      "trackers": [
        { "alias": "A598A0102D",
"mileage": 357.62 }
      ],
    },
    {
      "state": "QC",
      "trackers": [
        { "alias": "A598A0102D",
"mileage": 114.27 }
      ]
    }
  ]
}
```

# Geo-fence info

Return the configuration of a single geo-fence, including its shape (circle or polygon), schedule, associated trackers and entry / exit alarm flags.

**GET** /api/open/v1/region/{rid}

## PATH PARAMETERS

| PARAMETER | TYPE | DESCRIPTION                |
|-----------|------|----------------------------|
| rid       | int  | Region (fence) identifier. |

## RESPONSE FIELDS

### EXAMPLE REQUEST

**GET** /api/open/v1/region/112

### CIRCLE RESPONSE

```
{
  "status": 200,
  "message": "Operation is successful.",
  "data": {
    "shape": 0,
    "name": "test_accu2",
    "tids":
    ["e49e7e55fbe74aaab1c1f23f22892e0d"],
    "schedule": {
      "week": [
        { "hours": "", "dow": 0 },
        { "hours": "", "dow": 1 },
        { "hours": "", "dow": 2 },
        { "hours": "", "dow": 3 },
        { "hours": "", "dow": 4 },
        { "hours": "", "dow": 5 },
        { "hours": "", "dow": 6 }
      ],
      "hours": "",
      "start_time": 0,
      "type": 0,
      "end_time": 0
    },
    "uid":
    "7f81aaaf23a64dfc97fdf4add0da9fc5",
    "circle": {
      "radius": 4157.0,
      "latitude": 22.37441,
      "longitude": 113.56866
    },
    "id": 1979,
    "out_alert": 1,
    "enter_alert": 1
  }
}
```

| FIELD                                 | TYPE   | DESCRIPTION                                            |
|---------------------------------------|--------|--------------------------------------------------------|
| <code>data.uid</code>                 | string | Owning user id.                                        |
| <code>data.shape</code>               | int    | 0 = circle ·<br>1 = polygon.                           |
| <code>data.name</code>                | string | Fence name.                                            |
| <code>data.tids</code>                | array  | Tracker ids associated with the fence.                 |
| <code>data.schedule</code>            | object | Effective time window for the fence alarm.             |
| <code>data.schedule.type</code>       | int    | 0 = always on · 2 = one-time · 3 = daily · 4 = weekly. |
| <code>data.schedule.week</code>       | array  | Days of week (Sunday → Saturday) with active hours .   |
| <code>data.schedule.hours</code>      | string | Active hours, comma-separated 0 – 23.                  |
| <code>data.schedule.start_time</code> | int    | Schedule start — Unix UTC seconds.                     |
| <code>data.schedule.end_time</code>   | int    | Schedule end — Unix UTC seconds.                       |
| <code>data.out_alert</code>           | int    | Fence-exit alarm. 0 = off · 1 = on.                    |
| <code>data.enter_alert</code>         | int    | Fence-entry alarm. 0 = off · 1 = on.                   |
| <code>data.circle</code>              | object | Present when shape = 0 .                               |

#### POLYGON RESPONSE

```
{
  "status": 200,
  "message": "Operation is successful.",
  "data": {
    "shape": 1,
    "name": "test_region",
    "tids":
    ["e49e7e55fbe74aaab1c1f23f22892e0d"],
    "schedule": {
      "hours":
      "0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23",
      "start_time": 0,
      "type": 3,
      "end_time": 0
    },
    "uid":
    "7f81aaaf23a64dfc97fdf4add0da9fc5",
    "polygon": [
      { "latitude": 38.90378,
        "longitude": 109.7187 },
      { "latitude": 38.88962,
        "longitude": 109.74445 },
      { "latitude": 38.87599,
        "longitude": 109.72093 },
      { "latitude": 38.89216,
        "longitude": 109.69295 }
    ],
    "id": 1980,
    "out_alert": 1,
    "enter_alert": 1
  }
}
```

| FIELD                     | TYPE               | DESCRIPTION                                                                                                                    |
|---------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------|
|                           |                    | Contains<br><code>radius</code> ,<br><code>latitude</code> ,<br><code>longitude</code> .                                       |
| <code>data.polygon</code> | <code>array</code> | Present when<br><code>shape = 1</code> .<br>Ordered list of<br><code>{latitude,</code><br><code>longitude}</code><br>vertices. |

# Tracker note • read

Get the note attached to a single tracker. The same path also supports **POST**, **PUT** and **DELETE** for managing the note — documented in the next three sections.

**GET** /api/open/v1/trackers/{sn}/note

## PATH PARAMETERS

| PARAMETER | TYPE   | DESCRIPTION            |
|-----------|--------|------------------------|
| sn        | string | Tracker serial number. |

## RESPONSE FIELDS

| FIELD     | TYPE   | DESCRIPTION                        |
|-----------|--------|------------------------------------|
| data.note | string | Current note text for the tracker. |

### EXAMPLE REQUEST

**GET** /api/open/v1/trackers/3842402CD3/note

### 200 RESPONSE

```
{
  "status": 200,
  "message": "Operation is successful.",
  "data": {
    "note": "tracker note info"
  }
}
```



# Tracker note • add

Attach a new note to a tracker.

POST

/api/open/v1/trackers/{sn}/note

## BODY PARAMETERS

| PARAMETER | TYPE   | REQUIRED | DESCRIPTION |
|-----------|--------|----------|-------------|
| note      | string | required | Note text.  |

## RESPONSE FIELDS

| FIELD   | TYPE   | DESCRIPTION                            |
|---------|--------|----------------------------------------|
| status  | int    | HTTP status code mirrored in the body. |
| message | string | Human-readable result.                 |

REQUEST

POST /api/open/v1/trackers/3842402CD3/note

Content-Type: application/json

{  
 "note": "tracker note info"  
}

200 RESPONSE

{  
 "status": 200,  
 "message": "Operation is successful."  
}

# Tracker note • update

Replace the existing tracker note.

PUT

/api/open/v1/trackers/{sn}/note

## BODY PARAMETERS

| PARAMETER | TYPE   | REQUIRED | DESCRIPTION            |
|-----------|--------|----------|------------------------|
| note      | string | required | Replacement note text. |

## RESPONSE FIELDS

| FIELD   | TYPE   | DESCRIPTION                            |
|---------|--------|----------------------------------------|
| status  | int    | HTTP status code mirrored in the body. |
| message | string | Human-readable result.                 |

REQUEST

```
PUT /api/open/v1/trackers/3842402CD3/note
Content-Type: application/json

{
  "note": "modify tracker note info"
}
```

200 RESPONSE

```
{
  "status": 200,
  "message": "Operation is successful."
}
```

# Tracker note • delete

Remove the note attached to a tracker.

DELETE

/api/open/v1/trackers/{sn}/note

## PATH PARAMETERS

| PARAMETER | TYPE   | DESCRIPTION            |
|-----------|--------|------------------------|
| sn        | string | Tracker serial number. |

## RESPONSE FIELDS

| FIELD   | TYPE   | DESCRIPTION                            |
|---------|--------|----------------------------------------|
| status  | int    | HTTP status code mirrored in the body. |
| message | string | Human-readable result.                 |

### EXAMPLE REQUEST

DELETE /api/open/v1/trackers/3842402CD3/note

### 200 RESPONSE

```
{
  "status": 200,
  "message": "Operation is successful."
}
```

# Tracker alerts

Return all alarm events raised by a tracker over a time window — including motion, geofence, OBD-bus and AI driver-behaviour categories.

GET

/api/open/v1/trackers/{sn}/alerts

PATH PARAMETERS

| PARAMETER | TYPE   | DESCRIPTION            |
|-----------|--------|------------------------|
| sn        | string | Tracker serial number. |

QUERY PARAMETERS

| PARAMETER  | TYPE | REQUIRED | DESCRIPTION                                                  |
|------------|------|----------|--------------------------------------------------------------|
| start_time | int  | required | Query begin time — Unix UTC seconds.                         |
| end_time   | int  | required | Query end time — Unix UTC seconds. Defaults to current time. |

RESPONSE FIELDS

EXAMPLE REQUEST

GET /api/open/v1/trackers/3842402CD3/alerts?start\_time=1494864000&end\_time=1494950399

200 RESPONSE

```
{
  "status": 200,
  "message": "Operation is successful.",
  "data": [
    {
      "type": "Voltage low",
      "event_time": 1605490979,
      "latitude": 38.87599,
      "longitude": 109.72093,
      "note": "Terminal valtage low. valtage is 7992"
    }
  ]
}
```

| FIELD                          | TYPE                | DESCRIPTION                                   |
|--------------------------------|---------------------|-----------------------------------------------|
| <code>data[].type</code>       | <code>string</code> | Alert type — see the supported list below.    |
| <code>data[].event_time</code> | <code>int</code>    | Time the alarm was raised — Unix UTC seconds. |
| <code>data[].latitude</code>   | <code>float</code>  | Latitude at the time of the event.            |
| <code>data[].longitude</code>  | <code>float</code>  | Longitude at the time of the event.           |
| <code>data[].note</code>       | <code>string</code> | Supplementary detail (e.g. voltage reading).  |

SUPPORTED ALERT TYPES

The `type` field carries a human-readable string. The currently supported set is grouped by source below.

### General & motion

Battery low · Battery full · Disturbance · Movement · SOS · Speeding · Stopped · Idle · Long Stop · Over Driving · Over Local Speed · Rapid acceleration · Harsh braking · Voltage low · Communication interrupted · Maintenance · Emergency mode · ACC is abnormal · Illegal car use · collision · driver\_behavior · battery\_voltage\_low · Tracker lost power connection

### Geofence & POI

Entering geofence · Exiting geofence · Arriving POI · Leaving POI · Geofence alarm (OBD)

### OBD bus

Speeding (OBD) · Low voltage (OBD) · High engine coolant temperature (OBD) · Hard acceleration (OBD) · Hard deceleration (OBD) · Idle engine (OBD) · Towing (OBD) · High RPM (OBD) · Power on alarm (OBD) · Power off alarm (OBD) · Exhaust Emission (OBD) · Quick Lane change (OBD) · Sharp turn (OBD) · Fatigue driving (OBD) · Emergency alarm (OBD) · Crash alarm (OBD) · Tamper (OBD) · Illegal enter (OBD) · Illegal ignition (OBD) · OBD communication error (OBD) · Ignition on (OBD) · Ignition off (OBD) · MIL alarm (OBD) · Unlock alarm (OBD) · No card presented (OBD) · Dangerous driving (OBD) · Vibration alarm (OBD) · DTC for Passenger Car · DTC for Commercial Vehicle

### AI driver-behaviour

Lane departure warning (AI) · Distance closer warning (AI) · Forward Collision warning (AI) · Fatigue Driving Alarm (AI) · Smoking Alarm (AI) · Distraction Alarm (AI) · Driver Abnormal Alarm (AI) · Phone Call Alarm (AI)